

Переупорядочение памяти

Переупорядочение доступа к памяти (**Memory reordering**) это изменение порядка исполнения инструкций, заданных с помощью высокоуровневого языка программирования (**program order**). Переупорядочение происходит во время компиляции и во время выполнения программы на центральном процессоре. Переупорядочение доступа к памяти не должно изменять поведение однопоточной программы, чего нельзя сказать о многопоточных программах.

Чтобы нормализовать поведение многопоточной программы, логика которой может сильно пострадать от изменения порядка доступа к памяти, введены барьеры памяти, которые позволяют устанавливать строгую последовательность между обращениями к памяти до и после барьера. Обычно в большинстве многопоточных программах не встречаются барьеры памяти, однако используются примитивы синхронизации, такие как мьютексы, семафоры, условные переменные — по сути они включают в себя барьеры памяти не явно.

Барьеры компилятора

Компилятор может предпринять ряд оптимизаций. Например, регистры могут использоваться для кэширования значений переменных, что уменьшает количество обращений к памяти и ускоряет программу. Эти оптимизации не могут повлиять на логику работы однопоточного кода, но могут «сломать» код многопоточный.

Компилятор может попытаться применить оптимизацию для следующего фрагмента кода, где функции `foo` и `bar` запущены в разных потоках:

```
int a = 1;
```

```
void* foo(void* arg) { // CPU 0  
    while (a) ; return NULL;  
}
```

```
void* bar(void* arg) { // CPU 1  
    a = 0; return NULL;  
}
```

Если скомпилировать приведенный код без флагов оптимизации, то получим:

```
foo:
```

```
...
```

```
.L2:
```

```
    mov eax, DWORD PTR a[rip]
```

```
    test eax, eax
```

```
    jne .L2
```

```
    mov eax, 0
```

Видно, что вычитывается каждый раз глобальная переменная. Скомпилированный без флага оптимизации пример быстро завершится, то есть поведение программы будет ожидаемым.

Однако если посмотреть на работу этой функции со включенным флагом оптимизации (`-O2`), то можно заметить, что выполнение функции `foo` не завершится никогда:

```
foo:
```

```
...
```

```
    mov eax, DWORD PTR a[rip]
```

```
...
```

```
.L2:
```

```
    test eax, eax
```

```
    jne .L2
```

Очевидно, что оптимизация кеширования переменной в регистре не нарушит ход выполнения однопоточной программы, но в многопоточной программе такие оптимизации недопустимы.

Компилятор не предполагает, что этот код может быть многопоточным. Решение проблемы состоит в том, чтобы явно указать компилятору, где необходимо отменить оптимизации. Для этого можно использовать барьер компилятора. Это инструкция, которая не попадет в машинный код, но будет служить защитой от чрезмерной оптимизации.

Есть несколько способов реализовать барьер компилятора в Linux. Реализация на основе компилятора GCC определена в `include/linux/compiler-gcc.h`:

```
#define barrier( ) __asm__  
__volatile__( "" : : : "memory" )
```


Можно применить барьер к примеру и скомпилировать его с флагом оптимизации (-O2):

```
#define barrier() __asm__  
__volatile__("": : : "memory")
```

```
int a = 1;
```

```
void* foo(void* arg) {  
    while (a) barrier();  
    return NULL;  
}
```


Получений код теперь выглядит следующим образом:

```
foo:
```

```
...
```

```
.L3:
```

```
.L5:
```

```
    mov eax, DWORD PTR a[rip]
```

```
    test eax, eax
```

```
    jne .L3
```

После добавления `barrier( )` значение переменной считывается из памяти.

Протокол когерентности кешей

Операции с памятью CPU выполняются через кеш. Кеш CPU ядер требуется поддерживать в консистентном состоянии. Для этого используются специальные протоколы, например MESI. Протокол реализуется благодаря нескольким видам сообщений, вот некоторые из них:

Read (Чтение): это сообщение содержит физический адрес читаемой кэш-линии.

Read Response (Ответ на чтение): содержит данные, запрошенные предыдущим сообщением Read. Read Response может прийти от подсистемы памяти или от другого кэша. Например, если данные уже находятся в каком-либо кэше в состоянии `modified`, то такой кэш может ответить на Read.

Invalidate (Инвалидация): содержит физический адрес кэш-линии, которую требуется инвалидировать. Все прочие кэши должны удалить данные и ответить на это сообщение.

Invalidate Acknowledge (Подтверждение инвалидации): CPU, получивший Invalidate, должен удалить требуемые данные и подтвердить это сообщением Invalidate Acknowledge.

Read Invalidate (Чтение с инвалидацией): это сообщение содержит физический адрес читаемой кэш-линии. В то же время оно диктует другим кэшам удалить эти данные. По сути, это комбинация сообщений Read и Invalidate. Это сообщение требует ответа Read Response и множества Invalidate Acknowledge.

Writeback (Запись в память): содержит адрес и собственно данные для записи в память. Разрешает кэшам очистить соответствующую линию в

Существующие состояния MESI достигаются переходами, вот некоторые из них:
Переход a (M $\rightarrow$ E): Кэш-линия записана обратно в память, но CPU оставил её в кэше и имеет право изменять её.

Переход b (E $\rightarrow$ M): CPU пишет в кэш-линию, к которой имеет эксклюзивный доступ.

Переход c (M $\rightarrow$ I): CPU получает сообщение "read invalidate" кэш-линии в состоянии "Modified". CPU должен удалить свою локальную копию и ответить сообщениями "read response" и "invalidate acknowledge".

Переход d (I $\rightarrow$ M): CPU производит read-modify-write (RMW) операцию над данными, не находящимися в кэше. Он издает "read invalidate" сигнал, получая данные в "read response". CPU завершает переход только когда получит все ответы "invalidate acknowledge"

Переход e (S $\rightarrow$ M): CPU производит read-modify-write (RMW) операцию над данными, которые были read-only. Он должен инициировать "invalidate" сигнал и ждать, пока не получит полный набор ответов "invalidate acknowledge"

Переход f (M $\rightarrow$ S): Какой-то другой процессор читает данные, и эти данные находятся в кэше нашего CPU. В результате данные становятся read-only, что может привести к записи в память. Этот переход инициируется приемом сигнала "read". CPU отвечает сообщением "read response", содержащим запрашиваемые данные.

Переход g (E → S): Какой-то другой процессор читает данные, и эти данные находятся в кэше нашего CPU. Данные становятся разделяемыми и, следовательно, read-only. Переход инициируется приемом сигнала "read". CPU отвечает сообщением "read response", содержащим запрашиваемые данные

Переход h (S → E): CPU решает, что ему надо записать данные в кэш-линию, и поэтому посылает "invalidate" сообщение. Переход не завершается, пока CPU не получит полный набор ответов "invalidate acknowledge". Другие CPU выкидывают кэш-линию из своих кэшей сообщением "writeback", так что наш CPU становится единственным, кэширующим эти данные

Переход i (E → I): Другой CPU выполняет RMW-операцию с данными, которыми владеет наш CPU, так что наш процессор инвалидирует кэш-линию. Переход начинается с сообщения "read invalidate", наш CPU отвечает сообщениями "read response" и "invalidate acknowledge".

Переход j (I → E): CPU сохраняет данные в новую кэш-линию и передает сообщение "read invalidate". CPU не может закончить переход, пока не получит "read response" и полный набор "invalidate acknowledge". Как только запись завершится, кэш-линия переходит в состояние "modified" через переход (b)

Переход k (I → S): CPU загружает данные в новую кэш-линию. CPU посылает "read"-сообщение и завершает переход, получив "read response"

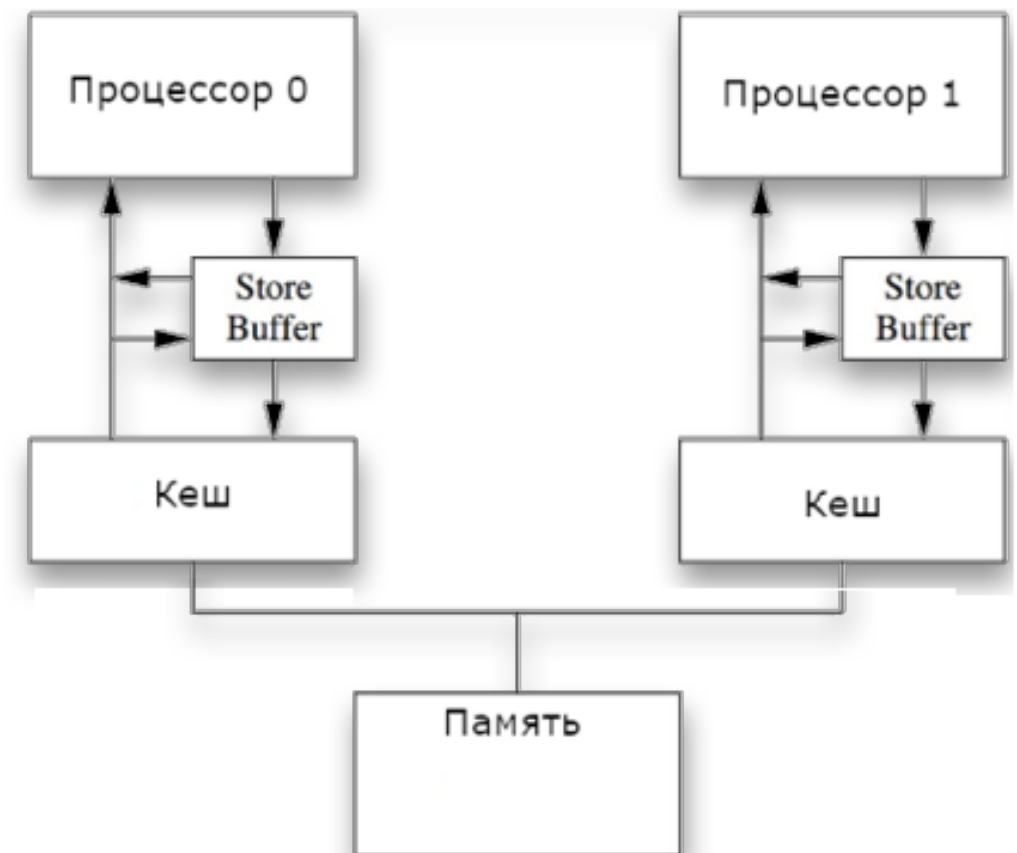
Переход l (S → I): Другой CPU хочет сохранить данные в кэш-линию, которая имеет статус read-only, так как какой-то третий CPU (или, например, наш) разделяет данные с нами. Переход начинается приемом "invalidate", и наш CPU отвечает "invalidate acknowledge"

Все операции с памятью CPU выполняет через кеш. В качестве мер борьбы с нежелательными простоями у каждого кеша процессора есть дополнительные буферы, аппаратные структуры данных, которые называются `store buffer` и `invalidate queue`. С `store buffer` CPU 0 может просто записать `write`-операцию и продолжать работу, пока кеш-линия не загружена из кеша другого процессора. После необходимого обмена сообщениями, когда кэш-линия наконец перейдет от CPU 1 к CPU 0, данные могут быть перемещены из `store buffer` в кэш-линию процессора 0. Использование `store buffer` приводит к дополнительным проблемам.

Нарушение самосогласованности

Для демонстрации проблемы, которую можно получить в системе, использующей `store buffer`, можно запустить следующие инструкции:

```
a = 1; //a = 0 в кеше CPU1  
b = a + 1; //b = 0 в кеше CPU0  
assert( b == 2 );
```



Переменные `a` и `b` имеют начальное значение 0. Переменная `a` находится в кэше процессора 1, а переменная `b` — в кэше процессора 0. Трудно представить, что может сработать `assert`, однако система использующая `store buffer` может выдать следующую последовательность событий:

1. CPU 0 начинает выполнять $a = 1$
2. CPU 0 ищет a в своём кэше, и видит, что его нет, — промах
3. Поэтому CPU 0 посылает сигнал `read invalidate`, чтобы получить исключительные права на кэш-линию с a
4. CPU 0 записывает новое значение a в свой `store buffer`
5. CPU 1 получает `read invalidate` — сообщение и отвечает на него, посылая кэш-линию с a и удаляя (`invalidate`) свою кэш-линию
6. CPU 0 начинает выполнять $b = a + 1$
7. CPU 0 получает ответ от CPU 1, который ещё содержит старое, нулевое значение a , и помещает линию в свой кэш
8. CPU 0 загружает a из своего кэша, — загружается значение ноль
9. CPU 0 выполняет сохраненный в `store buffer` запрос на запись в кэш, записывая в кэш-линию значение $a = 1$
10. CPU 0 добавляет единицу к нулю, прочитанному ранее в качестве a , и сохраняет результат в кэш-линии b (которая находится во владении CPU 0)
11. CPU 0 выполняет `assert(b == 2)` и выдает ошибку.

Проблема возникает из-за того, что система хранит две копии a — одну в кэше и вторую в `store buffer`.

Этот пример нарушает очень важную гарантию: каждый CPU должен всегда выполнять операции в том порядке, как они заданы в программе (так называемый `program order`). Для программистов эта гарантия является интуитивно понятным требованием, так что hardware-инженеры вынуждены были внедрить `store forwarding`: каждый CPU читает данные не только из своего кэша, но и обязательно из `store buffer`. Другими словами, каждая операция записи может прямо передаваться следующей операции чтения через `store buffer`, без обращения к кэшу. С учетом технических исправлений, шаг 8 в вышеприведенном примере должен прочесть корректное значение переменной `a` из `store buffer`. В результате значение `b` станет 2, что и требуется.

Нарушение глобального порядка памяти

Кроме того, из-за появления дополнительных буферов возможны проблемы, не решаемые с точки зрения hardware-инженеров. Можно рассмотреть следующие две функции, выполняемые на двух ядрах процессора:

```
void foo() { //CPU 0
    a = 1; // a = 0 изначально в кеше CPU1
    b = 1; // b = 0 изначально в кеше CPU0
}
```

```
void bar() { //CPU 1
    while ( b == 0 ) continue;
    assert( a == 1 );
}
```

Пусть CPU 0 выполняет `foo( )`, а CPU 1 — `bar( )`. Предположим, что в начале память, содержащая `a`, находится только в кэше CPU 1, а CPU 0 владеет памятью, содержащей `b`. Тогда возможна следующая последовательность действий:

1. CPU 0 выполняет `a=1`. Кэш-линия для `a` не находится в кэше CPU 0, так что CPU 0 помещает новое значение `a` в свой буфер записи и издает сигнал `read invalidate`.
2. CPU 1 выполняет `while (b==0) continue`, но `b` нет в его кэше, так что он посылает сообщение `read`
3. CPU 0 выполняет `b=1`. Он уже владеет `b` в своем кэше, то есть соответствующая кэш-линия находится в состоянии `exclusive` или `modified`, — так что он имеет полное право сохранить новое значение `b` в своем кэше, никому ничего не сообщая
4. CPU 0 получает сообщение `read` и передает в ответе кэш-линию со свежайшим значением `b`, попутно переводя эту линию в состояние `shared` у себя в кэше
5. CPU 1 получает кэш-линию с `b` и помещает её в свой кэш.
6. CPU 1 теперь может завершить выполнение `while (b == 0) continue`, так как он видит, что `b == 1`, и приступить к выполнению следующей инструкции
7. CPU 1 выполняет `assert(a == 1)`. Так как CPU 1 работает со старым значением `a`, условие не выполняется.
8. CPU 1 получает сообщение `read invalidate` и передает кэш-линию с `a` CPU 0, попутно инвалидируя эту линию в своем кэше. Но слишком поздно.
9. CPU 0 получает кэш-линию с `a` и выполняет запись из буфера (сбрасывает `store buffer` в свой кэш).

Чтобы поддержать взаимосвязь переменных в программе инженеры ввели инструкции барьеров памяти, с помощью которых программисты могут ограничить runtime кеширование.

Фрагмент программы должен быть изменен так:

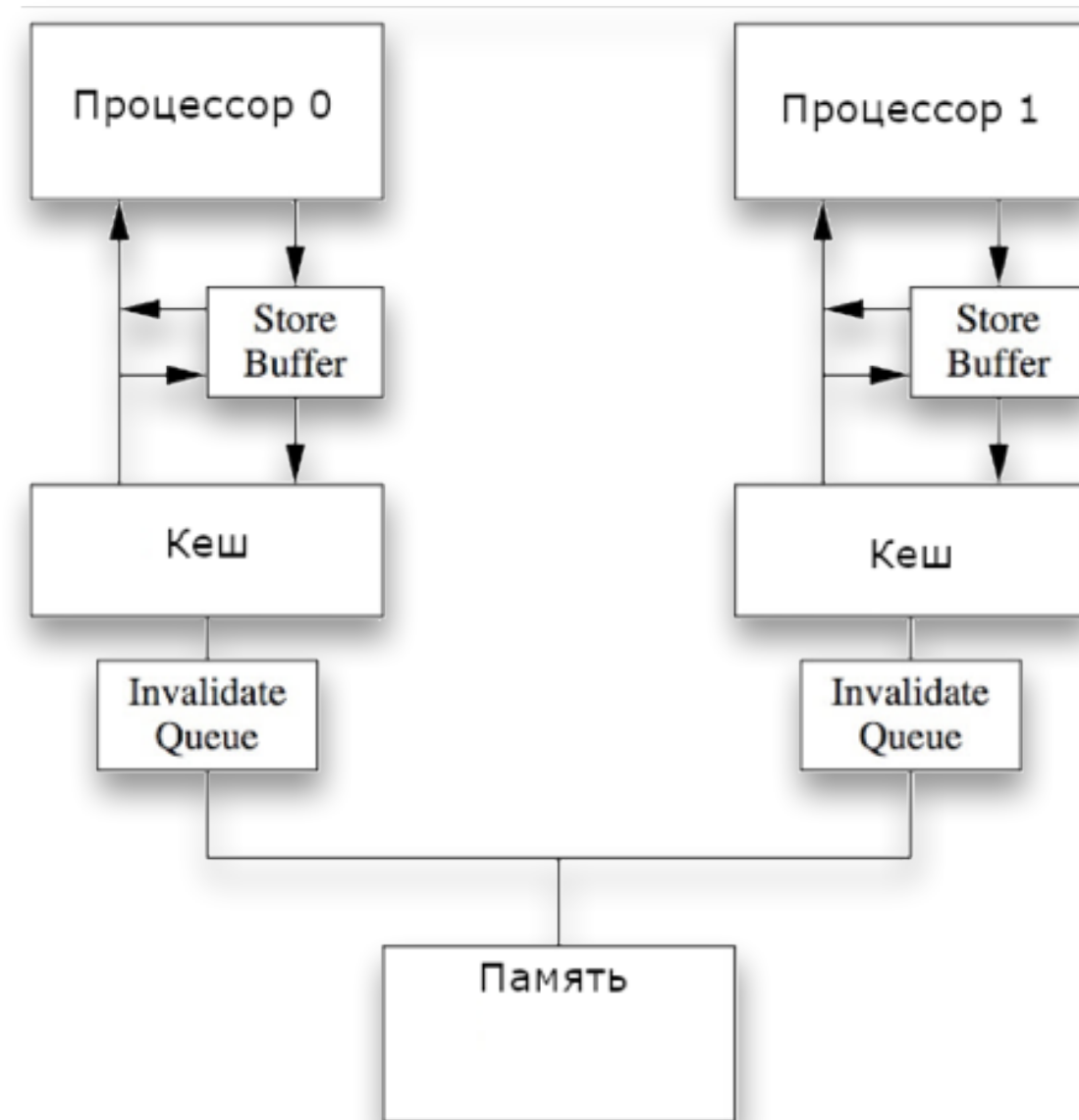
```
void foo() { //CPU 0
    a = 1; // a = 0 изначально в кеше CPU1
    smp_mb( ); // полный барьер памяти, в x86 это
инструкция MFENCE
    b = 1; // b = 0 изначально в кеше CPU0
}
```

```
void bar() { //CPU 1
    while ( b == 0 ) continue;
    assert( a == 1 );
}
```

Барьер памяти `smp_mb ( )` заставляет процессор сбросить `store buffer` перед выполнением следующей записи в кэш. CPU может либо просто остановиться, ожидая, пока не придут все запросы для его `store buffer`, и он не станет пустым. Тем самым в `store buffer` наводится некоторый FIFO-порядок.

Очередь инвалидации

К сожалению, каждый буфер записи технически имеет малое количество памяти. Это значит, что если CPU выполняет несколько записей, он рискует полностью заполнить свой `store buffer` (например, если каждая запись приводит к промаху и поиску кеш-линии). В этом случае CPU должен ждать, пока завершатся все инвалидации, чтобы затем он смог сбросить свой буфер в кэш и продолжить выполнение. Такая же ситуация может возникнуть сразу после барьера памяти, когда все последующие инструкции записи должны ждать завершения инвалидаций, вне зависимости от того, приводят ли эти записи к промахам кэша или нет. Такая ситуация может быть разрешена, если сообщения `invalidate acknowledge` будут обрабатываться быстрее. Один из путей достижения этого — ввести для каждого CPU очередь сообщений `invalidate`, или `invalidate queue`.



Процессор с `invalidate queue` может подтверждать `invalidate`-сообщения сразу, как только они будут появляться в очереди, вместо того, чтобы ждать, когда же кэш-линия инвалидируется. Помещение элемента в `invalidate queue` – это по сути обещание процессора обработать данное `invalidate`-сообщение перед тем, как отправлять любой сигнал протокола MESI, относящийся к данной кэш-линии. Буферизация `invalidate`-сигналов приводит к возможности нарушить порядок операций с памятью. Следующий пример являет одну из возможных проблем, возникающих с появлением очереди инвалидации.

Предположим, процессоры помещают `invalidate` - запросы в очередь и отвечают немедленно. Такой подход минимизирует задержку инвалидации кэша, но может нарушить барьер памяти, что видно в следующем примере.

Пусть переменные `a` и `b` изначально равны нулю и `a` находится в кэшах обоих процессоров в состоянии `shared`, а `b` находится во владении CPU 0 (то есть линия кэша в состоянии `exclusive` или `modified`).

Предположим, CPU 0 выполняет `foo()`, а CPU 1 — `bar()`:

```
void foo() { //CPU 0
    a = 1; //a = 0 shared
    smp_mb(); //полный барьер, для x86 MFENCE
    b = 1; //b = 0 в кеше CPU0
}
```

```
void bar() { //CPU 1
    while ( b == 0 ) continue;
    assert( a == 1 );
}
```


Последовательность операций может быть такой:

1. CPU 0 выполняет `a=1`. Соответствующая кэш-линия является `read-only` в кэше CPU 0, поэтому процессор помещает новое значение `a` в свой `store buffer` и посылает сообщение `invalidate`, чтобы CPU 1 сбросил свою кэш-линию с `a`
2. CPU 1 выполняет `while(b == 0) continue`, но `b` отсутствует в его кэше. Поэтому он посылает сообщение `read`.
3. CPU 1 получает `invalidate` от CPU 0, сохраняет его в свою `invalidate-очередь` и немедленно отвечает
4. CPU 0 получает ответ от CPU 1, поэтому он волен выполнить `smp_mb()` и поместить новое значение `a` из своего буфера записи в кэш
5. CPU 0 выполняет `b=1`. Он уже владеет кэш-линией с `b`, так что он просто сохраняет новое значение `b` в своем кэше
6. CPU 0 получает `read` - сигнал и передает кэш-линию с новым значением `b` процессору 1, заодно помечая эту линию как `shared`
7. CPU 1 получает кэш-линию с `b` и помещает её в свой кэш
8. Теперь CPU 1 может завершить `while(b == 0) continue` и перейти к следующей строке программы
9. CPU 1 выполняет `assert(a == 1)`. Так как в его кэше находится старое значение `a`, `assert` срабатывает и завершит программу

Несмотря на исключение, CPU 1 выполняет помещенное в очередь `invalidate`-сообщение и с опозданием инвалидирует кэш-линию «`a`».

В повышении быстродействия ответов на инвалидацию мало пользы, если это приводит к игнорированию барьера памяти. Поэтому барьеры памяти должны взаимодействовать с `invalidate queue`: когда процессор выполняет барьер памяти, он должен помечать все элементы в своей `invalidate` — очереди и притормозить все последующие чтения до тех пор, пока `invalidate` — очередь не будет им полностью обработана.

Барьеры чтения и записи

В последнем примере нет причин для того, чтобы `foo ( )` взаимодействовал с `invalidate`-очередью (так как нет чтения), равно как и нет причин для того, чтобы `bar ( )` взаимодействовал с буфером записи (так как нет записи).

Многие архитектуры предоставляют более слабые (и, как следствие, более быстрые) барьеры памяти, позволяющие упорядочить только чтение или только запись. Грубо говоря, барьер чтения (`read memory barrier`) взаимодействует только с `invalidate queue` (помечает её элементы, то есть наводит некоторый порядок в очереди), а барьер записи (`write memory barrier`) – только со `store buffer` (также помечает его элементы, наводит порядок в буфере). Полный барьер взаимодействует и с тем, и с другим.

Эффект этих полубарьеров таков: барьер чтения упорядочивает только чтения (`loads`) для процессора, который выполняет барьер. Все чтения перед барьером полностью завершаются, и только тогда начинают выполняться чтения после барьера. Аналогично, барьер записи упорядочивает только запись (`stores`) для своего процессора: все записи до барьера завершаются, и только потом начинают выполняться записи (`stores`) после барьера. Полный барьер упорядочивает чтения и записи, но опять, — только для процессора, выполняющего этот барьер.

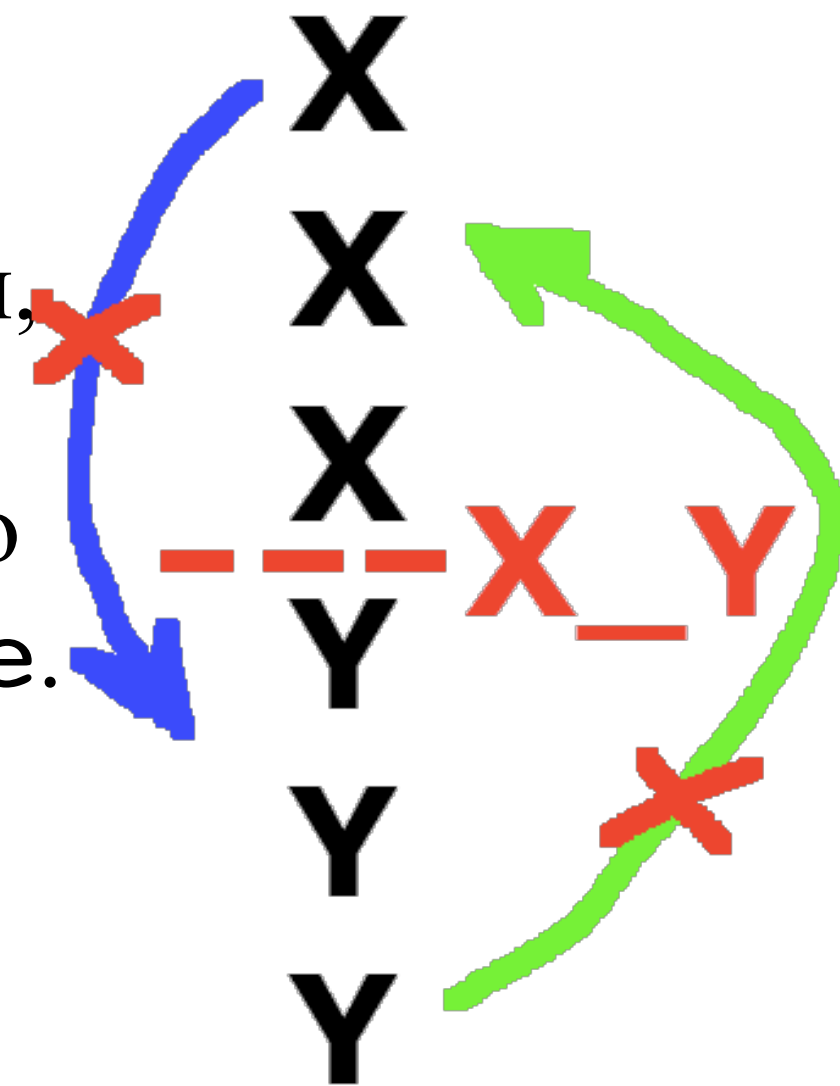
Если обновить пример так, что `foo` и `bar` будут использовать барьеры чтения/записи, получается следующее:

```
void foo() { //CPU 0
    a = 1; // a = 0 shared
    smp_wmb(); // барьер записи, для x86 SFENCE
    b = 1; // b = 0 в кеше CPU0
}

void bar() { //CPU 1
    while ( b == 0 ) continue;
    smp_rmb(); // барьер чтения, для x86 LFENCE
    assert( a == 1 );
}
```

Теоретические барьеры.

Существует множество архитектур процессоров, и не у всех из них есть `store buffer/invalidate queue`. Для решения проблем синхронизации договорились оперировать теоретическими, абстрактными барьерами памяти в виде `X_Y`. При этом `X` и `Y` могут принимать одно из двух значений – либо `Load`, либо `Store`. Смысл барьера – операции типа `X` гарантированно выполнятся до всех операций типа `Y`. То есть не может быть ситуации, когда операции типа `X` спустятся ниже барьера, или не может быть ситуации когда операции типа `Y` поднимутся выше барьера.



Выделяют четыре вида теоретических барьеров памяти:

LoadLoad

LoadStore

StoreLoad

StoreStore

Например барьер `StoreLoad` гарантирует что все операции записи закончатся раньше, чем начнутся операции чтения.

Если использовать терминологию абстрактных барьеров, то пару барьеров `LoadLoad/LoadStore` назвали `acquire`. А `LoadStore/StoreStore` – `release`.

В качестве аналогии можно представить критическую секцию, защищенную мьютексом.

Захват: Когда поток захватывает блокировку, он должен увидеть все изменения, сделанные предыдущим потоком в этой критической секции. Он как бы "приобретает" (*acquire*) все знания о состоянии защищенных данных.

Освобождение: Когда поток освобождает блокировку, он должен гарантировать, что все сделанные им изменения внутри секции станут видимы другим потокам. Он "отпускает" (*release*) эти изменения во внешний мир.

Модель памяти реальных процессоров:

1. `Sequential Consistency` – все операции чтения/записи по умолчанию применяет все барьеры памяти, как если бы в процессоре отсутствовали `Store Buffer` и `Invalidation Queue`.
2. `Strong-ordered` или `Total store-ordered (TSO)` – применяет `acquire` и `release` барьеры для любых операций чтения/записи, например – `x86 / 64`.
3. `Weak-ordered` – все необходимые барьеры нужно применять вручную, например `ARMv7`.
4. `Super-weak` – все необходимые барьеры нужно применять вручную + нет гарантий на то, что взаимозависимые строки кода не будут переупорядочены например `Alpha`.